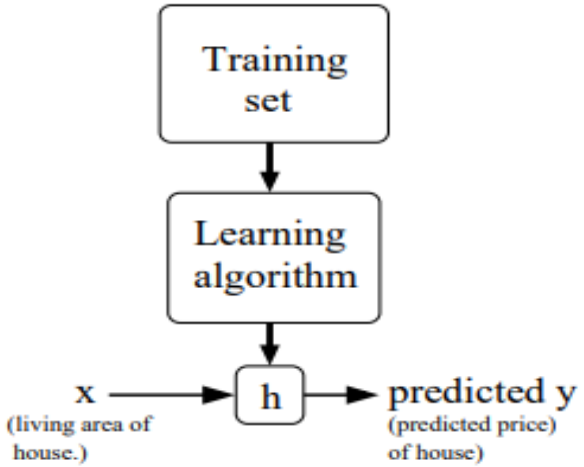
Machine Learning – CDS201

Module 2: Regression Models

Supervised learning



- Our goal in supervised learning :
  - Given a training set, to learn a function ***h : X → Y*** so that ***h(x)*** is a "good" predictor for the corresponding value of ***y***. For historical reasons, this function ***h*** is called a hypothesis.
  - When the target variable ***y*** that we're trying to predict is **continuous**, such as in our housing example (shown in the figure), we call the learning problem a **regression problem**. When ***y*** can take on only a small number of discrete values (such as if, given the living area, we wanted to predict if a dwelling is a house or an apartment, say), we call it a ***classification problem***.

Linear Regression

Given : Height of a Person (x)

Predict : Weight (Y) .

Height - x - Independent Variable

Weight - Y - Dependent Variable

Dependent on "x"

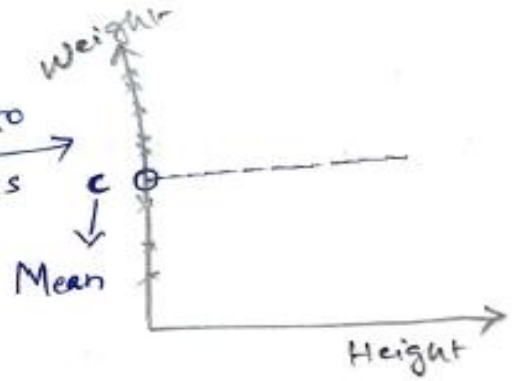
TALLER People tend to weigh More.

"Some Relationship EXists". HYPOTHESIS

Regression → DISCOVER THE RELATIONSHIP.

Start: No Relation $x = 0$ (Best Way to Get Weight ⇒ Avg)

Shift to Y-Axis →



Simplest Relation $\hat{y} = mx + c$

$\hat{y}$ → Predicted Value.

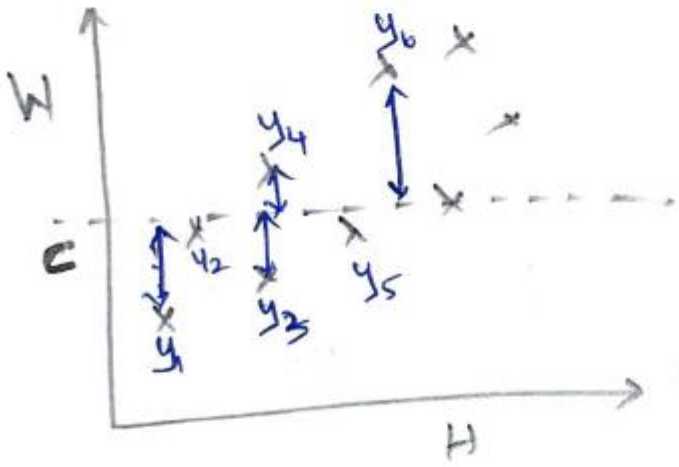
m → Slope

c → Intercept

Actual Value = y

Residue → Error = Actual − Predicted

$$\text{Error} = y - \hat{y}$$



$$\text{Error} = (y_1 - c) + (y_2 - c) + (y_3 - c) + \cdots$$

Sometime $y_1 \rightarrow +ve$ $y_2 \rightarrow -ve.$ chances of CANCEL OUT

⇒ BAD LINE → ZERO TOTAL ERROR

Least squares method of finding the optimal value

SUM OF SQUARE ERRORS [SSE]

$$SSE = \sum_{i=1}^{n} (y_i - c)^2.$$

$$SSE = (y_1 - c)^2 + (y_2 - c)^2 + (y_3 - c)^2 + \dots$$

Best Fit Line Aim → Zero Error (To start)

Practical → Min Error.

Identify Best "m" & "c"

"m" ↓ ROTATION

"c" ↓ Line UP/DOWN

CLOSED FORM SOLUTION Minimum of Curve.

$$SSE \begin{cases} M \\ c \end{cases} \quad \frac{\partial SSE}{\partial m} = 0 \quad \frac{\partial SSE}{\partial c} = 0$$

Page 8

$$SSE = \sum_{i=1}^{n} y_i - (mx_i + c)]$$

$$\frac{\partial SSE}{\partial c} = \sum_{i=1}^{n} 2\,(y_i - mx_i - c)\,(-1) = 0$$

$$\sum_{i=1}^{n} [y_i - mx_i - c] = 0$$

$$\sum_{i=1}^{n} y_i = m\sum_{i=1}^{n} x_i + nc$$

$\sum_{i=1}^{n} c \rightarrow$ Adding 'c' n times

$\div$ by n, $\quad \bar{y} = m\bar{x} + c$

$$\bar{y} = \frac{\sum y_i}{n}$$

$$\boxed{c = \bar{y} - m\bar{x}}$$

" Line Passes through $\boxed{\text{Mean}}$ to be optimum ".

$$\frac{\partial SSE}{\partial m} = \sum 2 (y_i - mx_i - c)(-x_i)$$

$$\sum x_i (y_i - mx_i - c) = 0 .$$

$$\sum x_i y_i - m \sum x_i^2 - c \sum x_i = 0 \quad \text{—} \; (1)$$

You know $c = \bar{y} - m\bar{x}$ sub in (1)

$$\sum x_i y_i - m \sum x_i^2 - (\bar{y} - m\bar{x}) \sum x_i = 0$$

Rearrange,

$$\left[\sum x_i y_i - \bar{y} \sum x_i\right] = \left[m\left(\sum x_i^2 - \bar{x} \sum x_i\right)\right]$$

You know $\bar{x} = \frac{\sum x_i}{n} \Rightarrow \sum x_i = n\bar{x}$

LHS

$$\sum x_i y_i - n\bar{x}\bar{y} \rightarrow \sum (x_i y_i - \bar{x}\bar{y})$$

RHS

$$m\left(\sum x_i^2 - n\bar{x}^2\right) \longleftrightarrow m \sum (x_i^2 - \bar{x}^2)$$

$$\sum (x_i y_i - \bar{x}\bar{y}) = m(x_i^2 - \bar{x}^2)$$

Rearrange

$$m = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

$$= \frac{Cov(x, y)}{Var(x)}$$

If $Cov(x, y) = 0 \Rightarrow m = 0 \Rightarrow$ Model predict Average ✗

$\hat{y} = c = \bar{y}$

HEIGHT for Everyone (Starting Point) ✗

HEIGHT PROVIDES NO INFORMATION ABOUT WEIGHT

- So the only possible way to predict weight is to take average across all weight data points.

If cov(x,y)=0

- The feature that I chose “height” is not a good assumption for me to predict “weight” . (BAD FEATURE) (Hypothesis fails)

- The entities are not related.

- Height doesn’t help me in predicting weight.

- ***Its like changing the side mirror of the auto if the auto engine is not working.***

Least squares method of finding the optimal value

If $Cov(x, y) \neq 0 \Rightarrow M \neq 0$ $\hat{y} = mx + c$

LINEAR RELATIONSHIP EXISTS BET X & Y

Height & Weight Move together either ↑ or ↓

$M = \frac{Cov(x, y)}{V(x)}$ $c = \bar{y} - m\bar{x}$ (Exact & Deterministic)

CLOSED-FORM SOLUTION

What is a Cost Function?

- It is a **function** that measures the performance of a model for any given data. **Cost Function** quantifies the error between predicted values and expected values and presents it in the form of a single real number.
- Cost is the Average loss over the dataset.
- Mean Squared Error (MSE)

$$J(w,b) = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$$

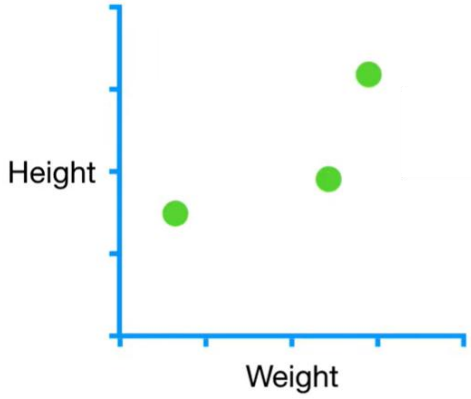
  - w – slope (weights), b – intercept (bias)

Gradient descent - Finding the best fit line

- Gradient descent is an optimization algorithm used in **machine learning** to minimize the cost function by iteratively adjusting parameters in the direction of the negative gradient, aiming to find the optimal set of parameters.
- Used to estimate model parameters by iteratively taking steps that minimize the loss function, such as the sum of squared residuals.
- Used in:
  - Linear regression
  - Logistic regression
  - Neural networks
  - Deep learning
- It is the core learning mechanism.

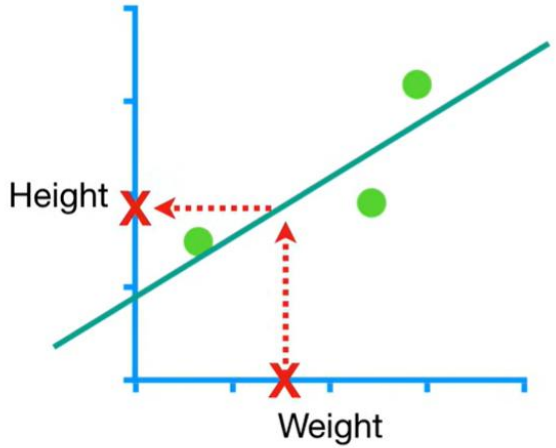
Let’s understand Gradient descent

- Simple dataset: x-axis = weight, y-axis = height



- Goal: fit line to predict height from weight
  - (e.g., weight 1.5 → height 1.9)

- Two-phase approach:
  - Phase 1: Find intercept only (slope fixed at 0.64 from least squares)
  - Phase 2: Find both intercept and slope simultaneously

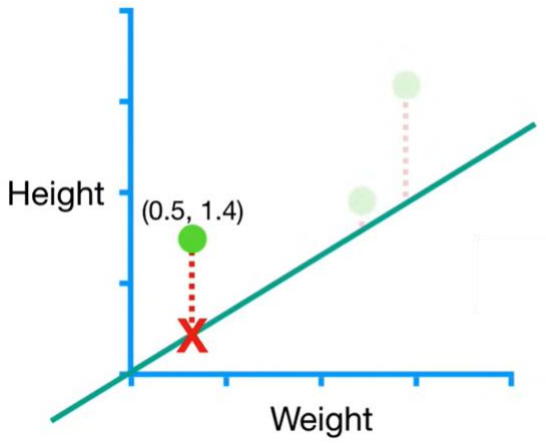


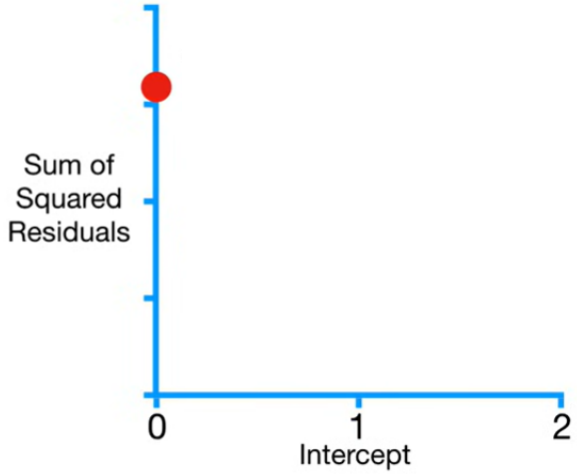
Phase 1: **Finding Intercept Only**

- Initial Setup
  - Fixed slope = 0.64 (by least squares estimate)
  - Random initial intercept = 0
- Loss function: sum of squared residuals
- Example calculation with 3 data points:
  - Point 1: weight 0.5, height 1.4
  - Point 2: weight 2.3, height 1.9
  - Point 3: weight 2.9, height 3.2

- Sum of squared residuals:
  - Predicted height (y) =

    [slope of the predicted line (m) * weight (x)]+ Intercept of the line (c)
  - Residue (error) = Actual height – predicted height
  - For the first data,
    - Predicted height = (0.64*0.5) + 0
    - Residue = 1.4 – 0.32 = 1.1
    - Do the same for the rest of the data points.
    - Point 1: weight 0.5, height 1.4 → predicted 0.32 → residual 1.1
    - Point 2: weight 2.3, height 1.9 → residual 0.4
    - Point 3: weight 2.9, height 3.2 → residual 1.3

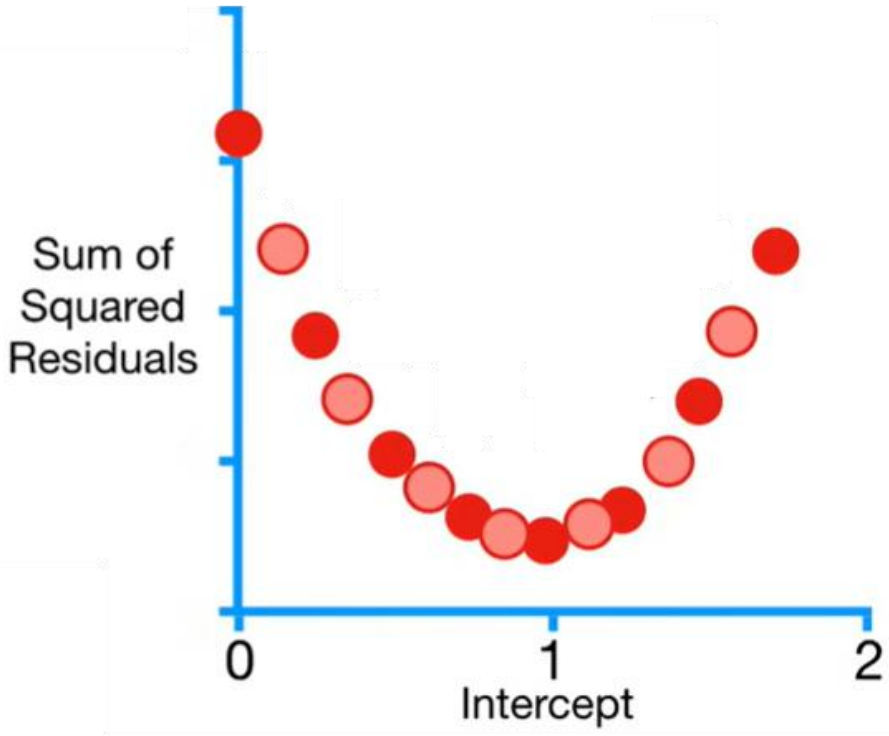




Sum of squared residuals = $1.1^2 + 0.4^2 + 1.3^2 = 3.1$

- By plugging a bunch of more intercept values, we will get the minimum sum of squared residuals. (slow and painful method)

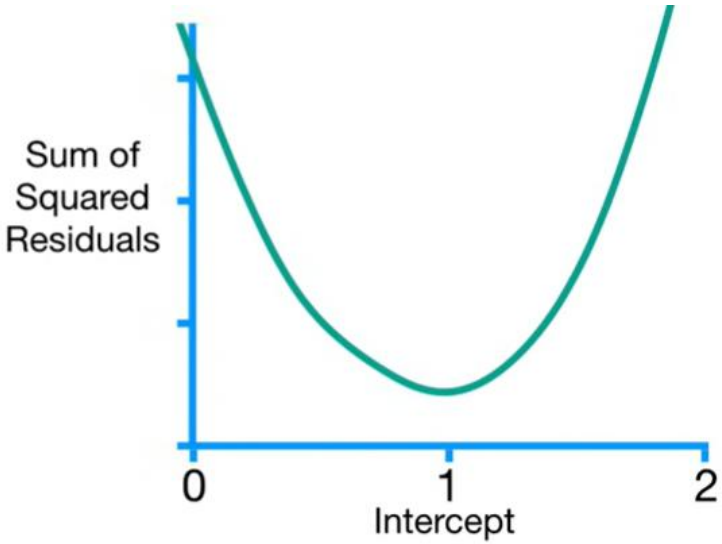
- Gradient descent will make it simple.
  - Few calculations far from the optimal solution
  - More calculations near to the optimal solution



- Gradient Descent vs Brute Force

- Brute force: test many intercept values, find minimum
- Gradient Descent: efficient approach using derivatives
  - Takes big steps when far from optimum
  - Takes small steps when close to optimum

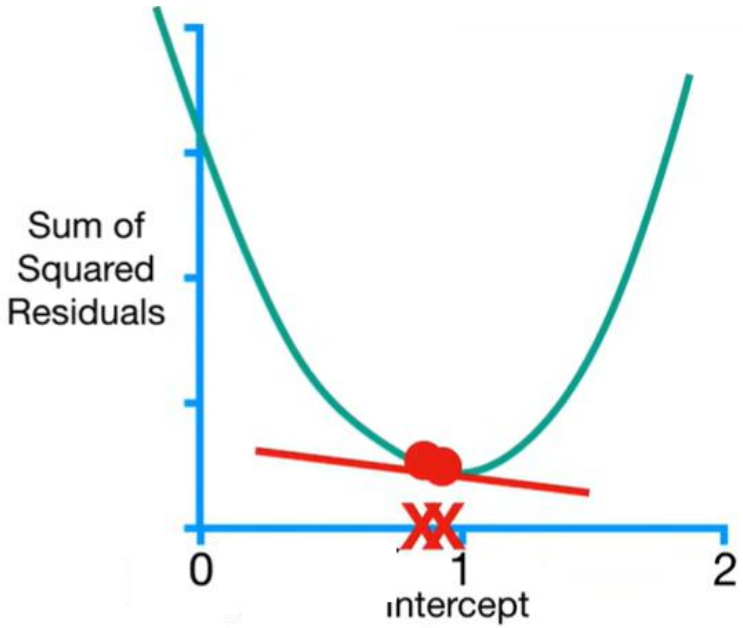
- Equation of the curve,



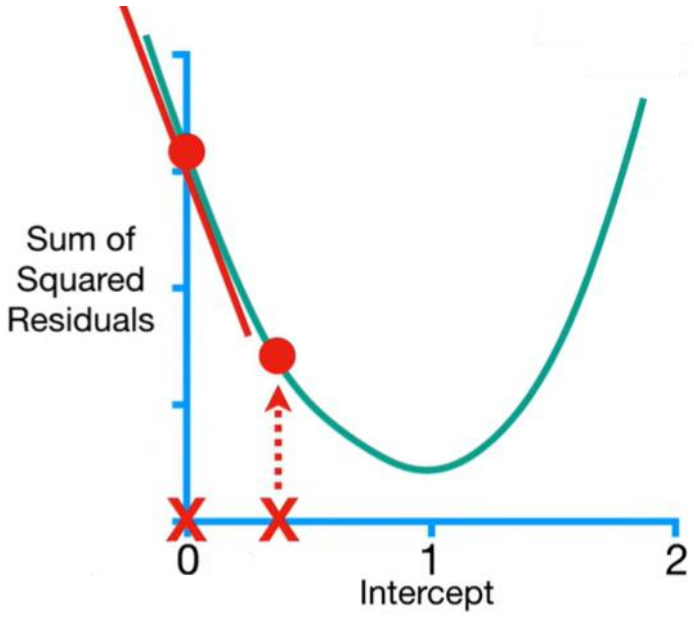
$$\text{Sum of squared residuals} = (\mathbf{1.4} - (\text{intercept} + 0.64 \times \mathbf{0.5}))^2$$
$$+ (\mathbf{1.9} - (\text{intercept} + 0.64 \times \mathbf{2.3}))^2$$
$$+ (\mathbf{3.2} - (\text{intercept} + 0.64 \times \mathbf{2.9}))^2$$

- Derivative of the sum of squared residuals w.r.t intercept
  - Applied chain rule to get derivative with respect to intercept
  - Final derivative:

$$-2(1.4-0.5\times intercept-0.64\times 0.5) - 2(1.9-intercept-0.64\times 2.3) - 2(3.2-intercept-0.64\times 2.9)$$



- Closer the slope of curve equal to zero take baby steps.

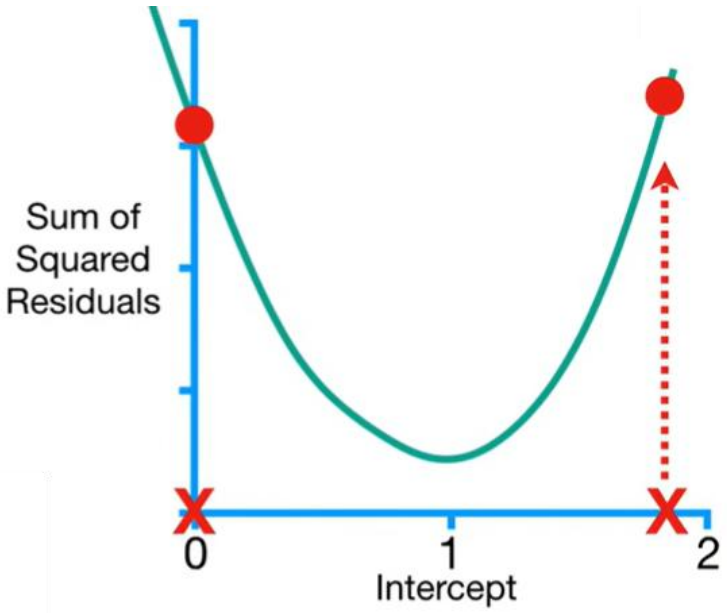


- Far away from the optimal value take larger steps.

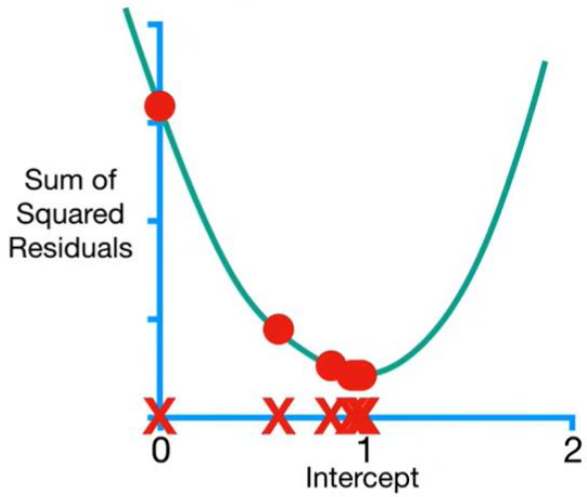
- If we take huge steps then we will increase the sum of squared residuals.

- If we take small steps we will reach the optimal value after several computational iterations.

- Size of the step should be related to the slope baby step , or big step (but again not too big).



- Step-by-Step Optimization
  - Initial intercept = 0 → derivative = -5.7
  - **Learning rate = 0.1**
  - Step size = slope × **learning rate** = -5.7 × 0.1 = -0.57
  - New intercept = old intercept - step size = 0 - (-0.57) = 0.57
- Subsequent steps:
  - Step 2: intercept = 0.8, step size = -0.23
  - Step 3: intercept = 0.89, step size = -0.09
  - Steps 4-6: intercept progresses to 0.95
- Final result: intercept = 0.95 (matches least squares)



- Stopping Criteria
  - Stop when step size < 0.001 (or smaller threshold)
  - Example: slope = 0.009, learning rate = 0.1 → step size = 0.0009 < 0.001 → stop

  - Maximum step limit: 1000+ steps as backup

Phase2: Finding Both Intercept and Slope

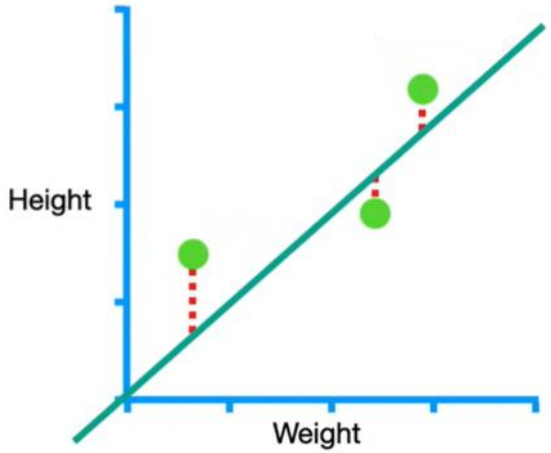
- 3D Loss Function
  - Loss function surface with intercept, slope, and sum of squared residuals axes
  - Need partial derivatives for both parameters (gradient)

- Gradient Calculation
  - Partial derivative with respect to intercept: same as phase 1
  - Partial derivative with respect to slope:
    - Applied chain rule, treating intercept as constant
    - Includes weight values (0.5, 2.3, 2.9) in derivative terms
- Two derivatives together = gradient

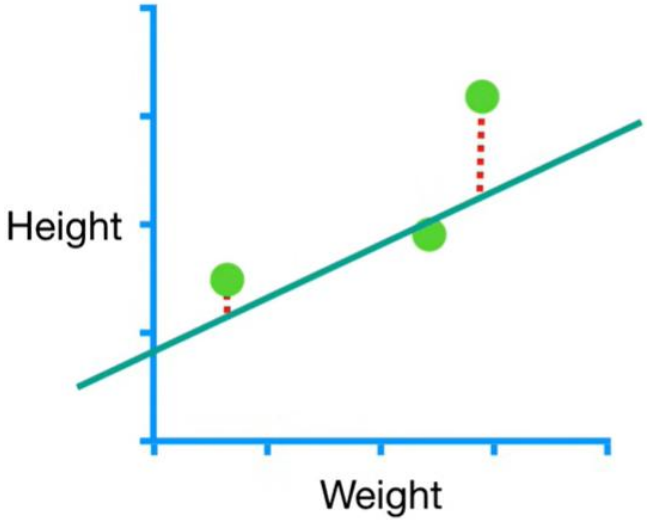
$$\frac{d}{d\ intercept}\text{Sum of squared residuals} = -2(\mathbf{1.4} - (\text{intercept} + \text{slope} \times \mathbf{0.5}) + -2(\mathbf{1.9} - (\text{intercept} + \text{slope} \times \mathbf{2.3})) + -2(\mathbf{3.2} - (\text{intercept} + \text{slope} \times \mathbf{2.9}))$$

$$\frac{d}{d\ slope}\text{Sum of squared residuals} = -2 \times \mathbf{0.5}(\mathbf{1.4} - (\text{intercept} + \text{slope} \times \mathbf{0.5})) + -2 \times \mathbf{2.9}(\mathbf{3.2} - (\text{intercept} + \text{slope} \times \mathbf{2.9})) + -2 \times \mathbf{2.3}(\mathbf{1.9} - (\text{intercept} + \text{slope} \times \mathbf{2.3}))$$

- Two-Parameter Optimization
  - Initial values: intercept = 0, slope = 1
  - Learning rate = 0.01 (smaller than earlier due to sensitivity)
  - Calculate two step sizes simultaneously
  - Update both parameters each iteration

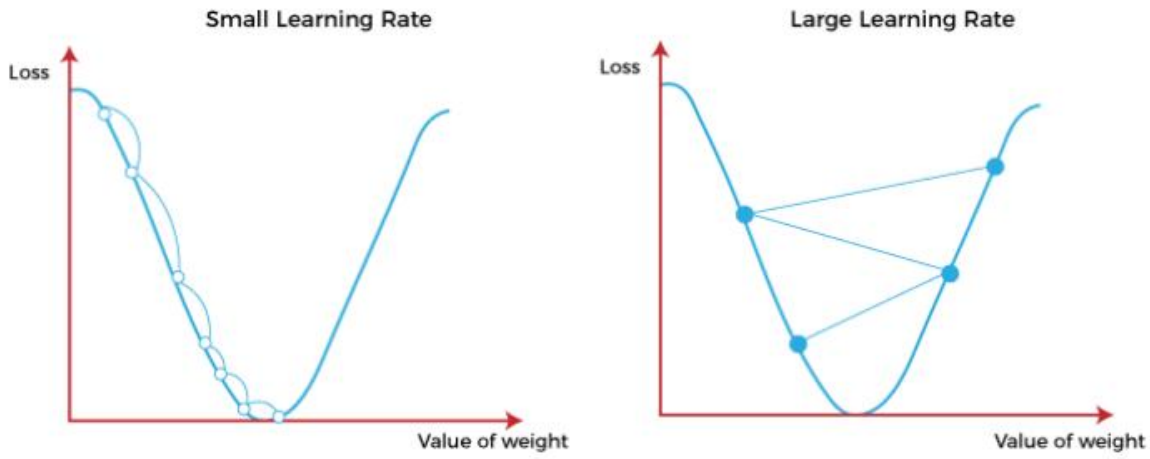


- Final result: intercept = 0.95, slope = 0.64 (here, it matches least squares)



- Gradient descent optimizes slope and intercept (two parameters)

- **Learning Rate Sensitivity**
  - The learning rate determines how large a step the algorithm takes toward the optimal solution.
  - **If too small**: Convergence is extremely slow, requiring many iterations to reach the minimum.
  - **If too large**: The algorithm may overshoot the minimum or diverge instead of converging.
  - Practice: start large, decrease with each step (automatic determination)
  - Critical parameter requiring careful tuning



Steps in GD algorithm

1. Take derivative of loss function for each parameter (compute gradient)
2. Pick random initial parameter values
3. Plug parameter values into gradient
4. Calculate step sizes (gradient × learning rate)
5. Calculate new parameters (old - step sizes)
6. Repeat steps 3-5 until convergence

- Scalability to Multiple Parameters
  - Method extends to any number of parameters
  - Just compute more partial derivatives
  - Same algorithm structure

- **Loss Functions and Applications**
  - Sum of squared residuals: one type of loss function
  - Many other loss functions for different data types
  - Gradient Descent works identically regardless of loss function choice

- In machine learning and moving forward,
  - Slope = WEIGHT (tilt of the line)
    - Eg: $y = 5x$ -> influence of x on y
  - Intercept = BIAS (shift of the line)
    - Eg: $y = 10$ -> y is independent of x

- In gradient descent, **local minimum, global minimum, and saddle point** are all places where the **gradient becomes zero.**
- The **global minimum** is the lowest value of the cost function across the entire parameter space. (Ideal solution)
- A **local minimum** is lower than nearby points, but not necessarily the lowest overall. (But better solution exists elsewhere)

Gradient descent – quick def

Gradient Descent is a key optimization algorithm in machine learning used to minimize cost functions by iteratively adjusting parameters.

Also known as **Batch gradient** as it runs over the entire dataset.

Check your understanding

- What's the role of gradient descent in linear regression?
  a) Gradient descent helps to determine what type of loss to use when training a model, for example, L_1 or L_2.
  b) Gradient descent is an iterative process that finds the best weights and bias that minimize the loss.
  c) Gradient descent removes outliers from the dataset to help the model make better predictions.

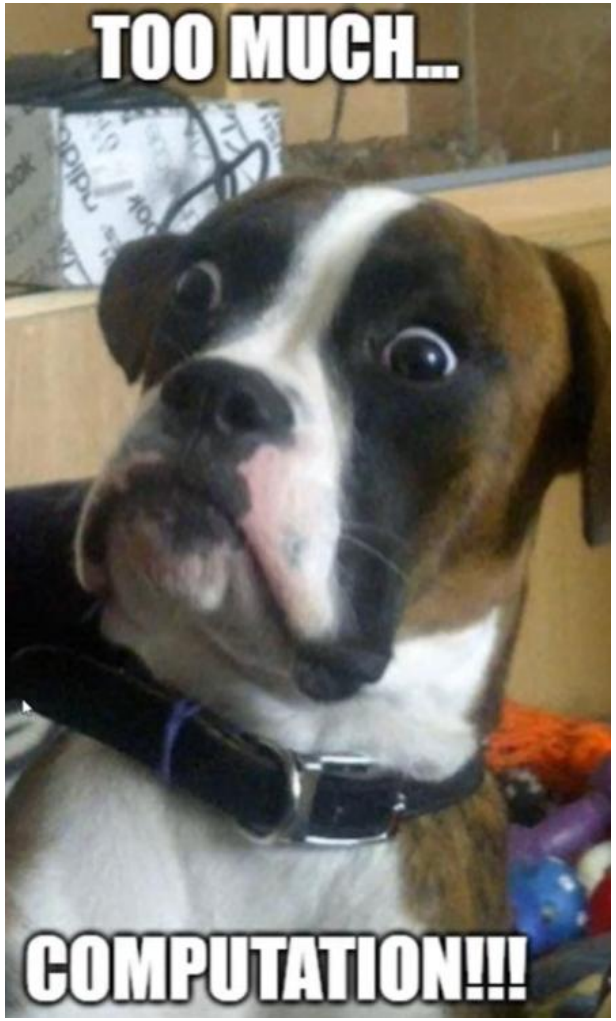
Now suppose:

- n = 1,000,000 samples
- d = 1000 features

- Operations per gradient step:
  - $1{,}000{,}000 \times 1000 = 1{,}000{,}000{,}000$
- That is: 1 billion operations per iteration

If you run 1000 iterations:

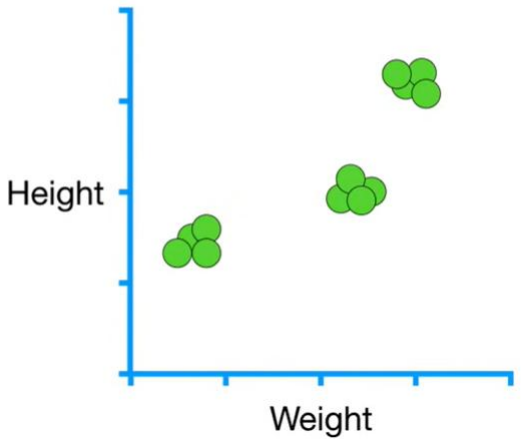
- $1{,}000{,}000{,}000 \times 1000 = 1{,}000{,}000{,}000{,}000$
- That is: 1 trillion operations
- This is expensive.



- Types of gradient descent:
  - Batch Gradient descent (all training samples)
  - Stochastic Gradient descent (One sample at random)
  - Mini-batch gradient descent (Subset of a data)

Stochastic gradient descent

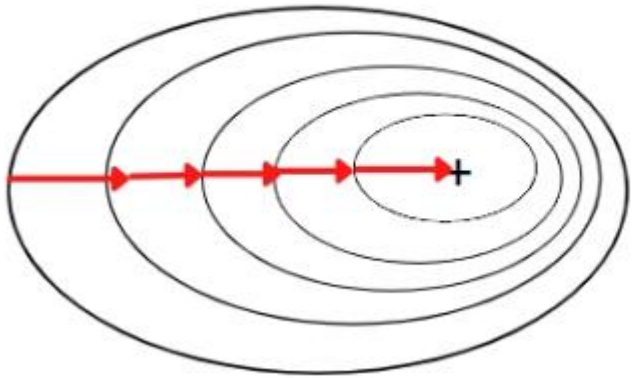
- ***SGD updates the parameters based on a randomly selected training example.*** This can lead to faster convergence because the updates are more frequent and noisy, but it can also result in more oscillations in the cost function due to the randomness of the updates.
- Faster than Batch Gradient Descent
- Introduces more noise in updates, which can cause fluctuations in the optimization path.



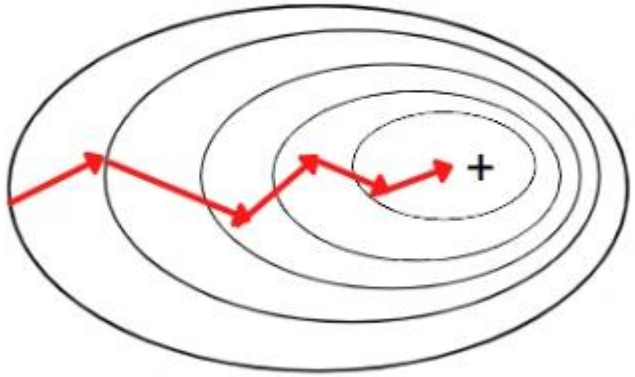
Mini-batch Gradient Descent

- Mini-batch gradient descent is a compromise between batch gradient descent and stochastic gradient descent.
- In mini-batch gradient descent, ***the gradients are computed on a small random subset of the training dataset, typically between 10 and 1000 examples, called a mini-batch***.
- This reduces the computational cost of the algorithm compared to batch gradient descent, while also reducing the variance of the updates compared to SGD.
- Mini-batch gradient descent is widely used in deep learning because it strikes a good balance between convergence speed and stability.

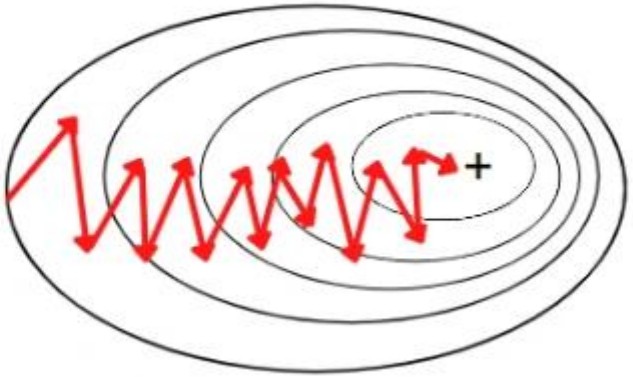
Batch Gradient Descent

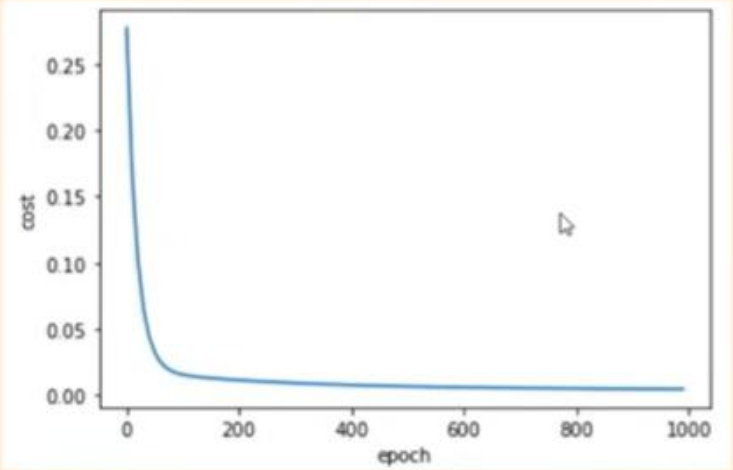
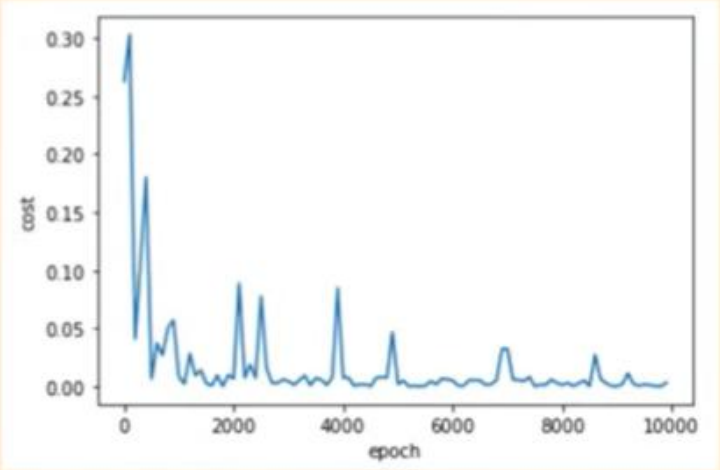


Mini-Batch Gradient Descent



Stochastic Gradient Descent



Batch Gradient Descent	Stochastic Gradient Descent (SGD)
Use **all** training samples for one forward pass and then adjust weights	Use **one** (randomly picked) sample for a forward pass and then adjust weights
Good for small training set	Good when training set is very big and we don't want too much computation
	

Batch Gradient Descent	Stochastic Gradient Descent (SGD)	Mini batch gradient descent
Use **all** training samples for one forward pass and then adjust weights	Use **one** (randomly picked) sample for a forward pass and then adjust weights	Use **a batch of** (randomly picked) samples for a forward pass and then adjust weights

- Quick recap:
  - Stochastic gradient descent is faster because it processes a single data point at a time, but mini-batch gradient descent provides a good balance of speed and stability, especially when training deep learning models.
  - Mini-batch gradient descent often converges faster, as it benefits from the accuracy of batch gradient descent and the speed of stochastic gradient descent.
  - Selecting the right gradient descent algorithm depends on the data size, computational resources, and the specific model requirements.